

Introduction to HTTP

Prof. Dr. Peter Braun

9. November 2024

1 Titlepage

Introduction to HTTP

Prof. Dr. Peter Braun



No Text – maybe some music :-)

2 Goals of this Unit

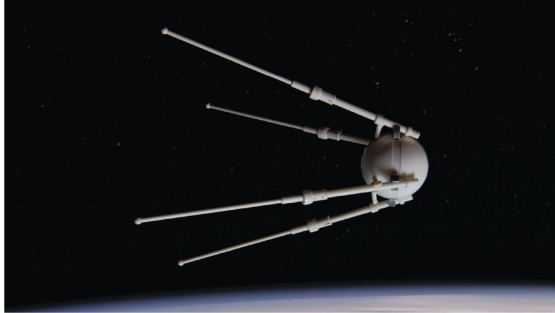
Learning Goals

- You learn about the history of the Internet
- You learn about Hypertext and Hyperlinks
- You learn about the structure of HTTP messages
- You learn about GET requests

Welcome to the unit, where we will briefly introduce HTTP. HTTP is the network protocol used by many client-server systems to communicate. But before we get into the technology, I would like to briefly tell you about the history of the Internet. After that, we will deal with a fundamental principle of the World Wide Web: setting links from one web page to another. After that, we will discuss some of the first basics of the HTTP protocol. And in particular, we talk about how the messages look like that are sent back and forth between client and server in a simple GET request.

3 Sputnik Crisis

Sputnik Crisis



- The USSR launched the first artificial earth satellite in 1957.
- The USA started several initiatives to close the technological gap.

The history of the Internet begins in 1957, the year the Soviet Union launched the first artificial satellite named Sputnik into space, causing a major shock in the technology scene in the United States. The scientists and technicians in the USA learned from the successful launch of the Sputnik satellite that they were no longer technological pioneers. Still, they had lost the race for the first successful satellite launch to the Soviet Union.

4 ARPANET

ARPANET



- ARPA = Advanced Research Projects Agency
- ARPANET started on 29 October 1969
- Transfer rate 50 kbit/sec via phone lines
- The term **Internet** was first mentioned in 1974 in RFC 675 about TCP

The shock was so deep that numerous initiatives were launched, right up to the highest political level, to return to the top of the world or become a technological development pioneer again—one of the initiatives also concerned computer science. In the following years, for example, they addressed the problem of how to connect individual computers via a network. ARPA, the Advanced Research Projects Agency, was founded for this purpose. After several years of research, the first successful network communication via telephone connections between two computers was demonstrated in Los Angeles and Stanford in 1969. In his film *Lo and Behold*, Werner Herzog documented the first message sent at the beginning. If you can access this film via one of the platforms, please watch the first five minutes. In the early years, the network was called ARPANET and quickly expanded. The term Internet was first mentioned in 1974.

5 Tim Berners-Lee, 1989

Tim Berners-Lee, 1989



- Distributed document management system
- Proposal in March 1989:
<https://www.w3.org/History/1989/proposal.html>
- He developed the first versions of HTML and HTTP
- ACM A. M. Turing Award in 2017

We are now skipping many years of development, during which the number of computers connected via the Internet grew rapidly. New communication protocols were developed; for example, the protocol for sending e-mails was also developed. It was not until 1989 that the HTTP communication protocol was finally developed. We will study the HTTP protocol intensively in this and the following units on programming client-server systems. Today, the British physicist Tim Berners-Lee is considered the most famous inventor of the so-called World Wide Web. Back in 1989, he was employed at the nuclear research center CERN in Switzerland and was concerned with the question of how documents could be made available in a simple manner via the Internet so that a rapid exchange, for example, of research results, would be possible within the community of scientists. Tim Berners-Lee proposed two technologies at that time. One was the HTML page description language, and the other was the HTTP communication protocol. In 2017, he received the ACM Turing Award for this invention, which is comparable to the Nobel Prize in different sciences.

6 World Wide Web

World Wide Web



- First Web server in November 1990
- First Web browser in March 1991

But what exactly is this World Wide Web? The World Wide Web is a distributed system. According to our understanding today, it's even the largest distributed system in the world. It consists of millions of web servers that provide all documents in HTML files. We will look at what such an HTML document looks like in a moment. To display an HTML page, we need a web browser. So we need two computers: one that provides the documents in HTML format, and this computer we call the webserver. And another computer that can display the HTML pages to humans using a web browser. The web server is software that makes the locally stored HTML files available to another computer over the network. This is very simple software for the first step. A request from a remote web browser arrives at a web server; this request must contain at least the document's name to be displayed on the web browser. The web server searches for this document in its locally stored files and sends it to the web browser over the network. The web browser is the client that sends a request to a server, receives the document from the web server, and displays it on the screen accordingly. You can already see the basic pattern of a client-server system here. As a client, the web browser always starts a request for the web server. Tim Berners-Lee wrote the first versions of a web browser and a web server. The first web server was ready in November 1990 and ran on a NeXT computer. In case you are not entirely familiar with the history of computer science in the late eighties, NeXT was the company Steve Jobs founded after he was fired from Apple. The NeXT computer was one of the most advanced computers at that time in terms of hardware and operating system. The first web browser, known as World Wide Web by intention, was released in March 1991.

7 Web Page Rendered by a Web Browser

Web Page rendered by a Web Browser

Hypertext

From Wikipedia, the free encyclopedia

For the concept in semiotics, see [Hypertext \(semiotics\)](#).

"Metatext" redirects here. For the literary concept, see [Metafiction](#).

Hypertext is text displayed on a [computer display](#) or other [electronic devices](#) with references ([hyperlinks](#)) to other text which the reader can immediately access, or where text can be revealed progressively at multiple levels of detail (also called [StretchText](#)).^[1] The hypertext pages are interconnected by hyperlinks, typically activated by a [mouse click](#), [keypress sequence](#) or by touching the screen. Apart from text, hypertext is sometimes used to describe tables, images and other presentational [content forms](#) with hyperlinks. Hypertext is the underlying concept defining the structure of the [World Wide Web](#),^[2] with pages often written in the [Hypertext Markup Language](#) (a.k.a. HTML). It enables an easy-to-use and flexible connection and sharing of information over the [Internet](#).

Note how hypertext is not just flat text with highlights or paragraphs omitted during display, but rather, the text is hyper-structured with hyperlinks or other structures embedded inside a page, including hidden search words, to control the display and connection with other pages or hypertext nodes.

Contents [hide]
1 Etymology
2 Types and uses of hypertext
3 History
4 Implementations
5 Academic conferences
6 Hypertext fiction
6.1 Forms of hypertext
6.2 Critics and theorists
7 See also
8 References
9 Bibliography
10 Further reading
11 External links
11.1 Hypertext conferences

Tim Berners-Lee proposed the HTML page description language as a simplification of technologies already existing at that time, such as SGML. The goal was to make HTML very easy, particularly for persons from fields other than computer science, to describe the content and partly the document's appearance. In the HTML page description language, such a document is a simple text file with specific commands; for example, a paragraph is marked or a heading. We will look at an example of this in a moment. The web browser can display files in page description language HTML accordingly on the screen so that the control commands for a heading are no longer displayed as commands but as a nicely formatted heading, for example, in a larger font or a different color. On the slide, you can see the presentation of the Wikipedia article about hypertext in a web browser. We have chosen this article not without reason but because we want to show a fundamental principle of HTML documents or the whole World Wide Web with this example: hyperlinks. Note the words written in blue in the article. Behind these words is a redirection to other Wikipedia articles or pages from the World Wide Web. You all know this: if you move the mouse over the word and click, another page opens in the web browser. We call this principle of forwarding hyperlinks. So these documents are linked together – we call this hypertext – so that when you surf the World Wide Web, as we call it, you are forwarded from one document to the next and possibly from one computer to the next. Because the hyperlink could also point to another document, just stored on another physical server. So, the computers are connected to a network, and the documents are connected. This network gives the World Wide Web its name.

8 Web Page is shown in HTML

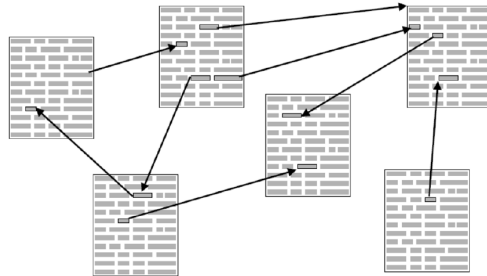
Web Page shown in HTML

```
<p><b>Hypertext</b> is text displayed on a <a href="/wiki/Computer_display" title="Computer display" class="mw-redirect">computer display</a> or other
<a href="/wiki/Electronic_devices" title="Electronic devices" class="mw-redirect">electronic devices</a> with references (<a href="/wiki/Hyperlinks"
title="Hyperlinks" class="mw-redirect">hyperlinks</a>) to other text which the reader can immediately access, or where text can be revealed
progressively at multiple levels of detail. (also called <a href="/wiki/StretchText" title="StretchText">StretchText</a>).<sup id="cite_ref-1"
class="reference"><a href="#cite_note-1"></span></span></span></a></sup> The hypertext pages are interconnected by hyperlinks, typically
activated by a <a href="/wiki/Mouse_(computing)" title="Mouse (computing)">mouse</a> click, keypress sequence or by touching the screen. Apart from
text, hypertext is sometimes used to describe tables, images and other presentational <a href="/wiki/Content form" title="Content form" class="mw-
redirect">content forms</a> with hyperlinks. Hypertext is the underlying concept defining the structure of the <a href="/wiki/World_Wide_Web"
title="World Wide Web">World Wide Web</a>.<sup id="cite_ref-2" class="reference"><a href="#cite_note-2"></span></span></span></a></sup> with
pages often written in the <a href="/wiki/Hypertext_Markup_Language" title="Hypertext Markup Language" class="mw-redirect">Hypertext Markup Language</a>
(a.k.a. HTML). It enables an easy-to-use and flexible connection and sharing of information over the <a href="/wiki/Internet"
title="Internet">Internet</a>.</p>
<p>Note how hypertext is not just flat text with highlights or paragraphs omitted during display, but rather, the text is hyper-structured with
hyperlinks or other structures embedded inside a page, including hidden search words, to control the display and connection with other pages or
hypertext nodes.</p>
```

Let's look at a document with the HTML markup language to understand what kind of control command is used here. At the very beginning, you see the command P, which is used to start a paragraph. After that comes the control command B, which indicates that everything after should be printed in bold; after the word Hypertext, you see the corresponding command to switch off the bold print; you can recognize it by the slash. At the beginning of the second line, you see control command A, which defines a hyperlink. Shortly after that comes the parameter HREF, and behind the equal sign is the document's address, which is referred to in quotation marks. In this example, the target document is located in a directory named "Wiki," and the document is called "Electronic Devices."

9 Hypertext

Hypertext



- Non-linear **documents** with **hyperlinks**
- Hypertext documents are written in HTML
- HTML became popular because it's simple

The World Wide Web works quite differently compared to a classic book. You would read a book usually from front to back. With textbooks, you might read only individual chapters. Documents on the World Wide Web are based on hypertext, i.e., you start with a document and then follow the hyperlinks to other documents. The possible applications for such hyperlinks are, of course, manifold. At first, you could now imagine creating a hyperlink from one place in a document to another place. By this, we could realize something like a table of contents in a book. A longer text is divided into chapters, each representing a separate document. A hyperlink would also lead from one document to another. For example, you could imagine that each occurrence of a technical term links to the place and the document in which the word was defined for the first time. Hyperlinks allow the reader to go deeper into the content. For example, the document offers the reader hyperlinks to get more information about a topic or a word. In this case, the hyperlinks would also point, for example, to documents written by other people and made available on different web servers. Thus, with hyperlinks, the linear reading of documents is complemented by the ability to jump to other places in a document or even other documents to get more information. The document gives the reader a new degree of freedom to decide what he wants to read. But it must be clear that this degree of freedom is limited. Only if the author of a document has provided such a hyperlink can the reader get to the linked information. The document's author provides the reader with the possibility for further details and leaves him the freedom to decide whether he wants to accept this offer. Unfortunately, this aspect of hypertext and hyperlinks is no longer consistently pursued on the websites published nowadays. Wikipedia is a positive exception here.

10 Hypertext is an Old Idea

Hypertext is an Old Idea

- Paul Otlet: Universal Library (1894)
- Vannevar Bush: As We May think: Memex (1945)
- Ted Nelson: Xanadu (1965)
- Bill Atkinson: Hypercard (1987)

By the way, the idea of hypertext is not an invention of Tim Berners-Lee but goes back to ideas more than 70 years ago. The inventor of the name "hypertext" is considered to be Ted Nelson, who has been working on his vision of an ideal hypertext system called Xanadu for many decades. Nelson is also one of the sharpest critics of today's World Wide Web, and one of his main criticisms is that hyperlinks on the World Wide Web are only a unidirectional connection. The link takes you from the source document to the target document but does not allow the reader, when reading a document, to jump to the documents that refer to that document, i.e., the source documents. To make that possible, the links would have to be bidirectional. This is implemented accordingly in his Xanadu system but not in the World Wide Web.

11 The Web – Summary

The Web – Summary

- The Web is the largest distributed system we know.
- Documents are provided by Web Servers.
- Documents are written in HTML, JavaScript, and CSS.
- Documents are transferred using HTTP.
- Web Clients display documents to the end user.
- Hyperlinks are used to drive users through the Web.
- **The Web follows the same design principles we will study in the next weeks.**

So, let's briefly summarize what we have learned so far about the World Wide Web. To our understanding, the World Wide Web is the largest distributed system in the world. It works strictly according to the principle of client and server, where the role of the client is taken by the web browsers and the role of the server by the so-called web servers, which provide documents. As described, hypertext's basic principle has yet to be consistently implemented in today's World Wide Web. You only have to look at a typical article on any news site, such as Spiegel.de, and see where hyperlinks are used. In the actual news text, you will hardly find any words pointing to other providers' documents in the sense of a hyperlink. What you will often find, of course, are links behind advertisements and links to documents from their platform. Over 30 years have passed since the invention of HTML as a page description language for documents. Accordingly, how documents are available on the Web has also changed significantly. The original controls for headings and paragraphs in HTML have been replaced by CSS, which is another way to define the layout of a page. To look at a good example of hypertext, look at the Wikipedia pages. There, hyperlinks are used to navigate within a document as well as between documents, and there are also links to further information, i.e., pages outside Wikipedia.

12 Overview

Overview

- HyperText Transfer Protocol (HTTP)
- Developed by **Tim Berners-Lee** in 1989
- **Roy Fielding** worked on later versions
- Based on TCP/IP for reliable communication
- Versions
 - HTTP/0.9 (1991)
 - HTTP/1.0 (1996)
 - HTTP/1.1 (1999)
 - HTTP/2 (2015)

In addition to the HTML page markup language, Tim Berners-Lee developed the first version of a network transfer protocol called HTTP about 30 years ago. HTTP stands for Hypertext Transfer Protocol and is a protocol on the so-called application layer, i.e., above TCP/IP. The HTTP protocol controls the communication between web browsers and servers. Besides Tim Berners-Lee, Roy Fielding was also involved in developing this protocol. We will encounter the name Roy Fielding even more often in the further course of this course because he later wrote down the basic concept or the architectural principles on which the World Wide Web is based as part of his dissertation and gave it the name REST. What REST means exactly, we will learn later. The HTTP protocol, somewhat surprisingly, has seen relatively few versions in the last 30 years. The first ten years saw version 0.9-1.1 development, and then it took until 2015 for the version HTTP 2.0 to be released. In this course, we will only deal with the basic principles of the HTTP protocol and will only marginally discuss the innovations of HTTP 2.0.

13 URI

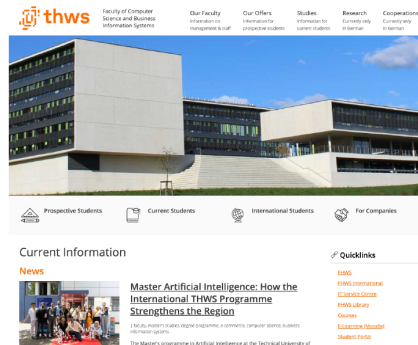
URI

- Uniform Resource Identifier
- `scheme://hostname[:port]/path/to/doc[?param]`
- Example: `http://example.org:8080/ping?text=hallo`
- URIs provide a means for namespaces by itself

Before a client can send a request to a web server using the HTTP protocol, we must first define what the global address of a document is. For this purpose, there is the concept of URLs, which stands for "Uniform Resource Locator." The newer and more correct term is "Uniform Resource Identifier," or URI. Such URI always starts with the so-called scheme name, which can be simplified as the protocol for transmitting via TCP/IP. If HTTP is to be used, it says HTTP. If a secure connection is to be used, it says HTTPS. However, the concept for URIs also allows, for example, FTP for a simple file transfer in the scheme. In the position for the hostname, there is either the IP address of a computer or the entire domain name. For example, the computer that provides our faculty's web pages is `fiw.fhws.de`. A port specification is optional; by default, port 80 is used for simple HTTP communication, and port 443 is used for encrypted communication. With this information about the scheme, the hostname, and the port, the client can select the transmission protocol and open a TCP/IP connection to the web server. In the URI, the slash is followed by a local path to the desired document, comparable to a hierarchical directory structure in a file system. The last element is then, for example, the name of a concrete file. Subsequently, additional parameters can be transferred. If this is used, it is introduced with the symbol question mark. It is followed by several parameters and values, where an equal sign separates them. Ampersands separate several such pairs. We will come back to this later.

14 Opening a Web Page in a Web browser

Opening a Web Page in a Web browser



- Hostname is resolved to IP address
- The browser opens a TCP/IP connection
- It sends an HTTP message and waits for a reply

Using an example, let's look at simple communication using the HTTP protocol. As a client, we use a web browser and aim to retrieve the web page for our faculty. In the browser's address bar, we enter the address `https://fiw.thws.de`; the web browser will first request our web server. However, the full name `fiw.thws.de` is translated into the IP address beforehand. Port 443 is chosen by default because we want to use an encrypted connection. A TCP/IP connection to the webserver is established with the IP address and the port. After establishing the connection, the web browser sends the first message regarding the HTTP protocol. The HTTP protocol consists of messages with a very simple structure and, at least in the basics, can be read and understood by humans without problems. A message sent from the client to the server is called a request. The message sent back from the server to the client is called a response.

15 Request sent by your browser

Request sent by your browser

```
1 GET /index.html HTTP/1.1
2 Host: fiw.thws.de
3 Accept: text/html
4 Cache-Control: no-cache
```

- The first line is named **request line** and contains of
 - ❶ HTTP verb, for example, GET
 - ❷ Document path, for example `/index.html`, and
 - ❸ Protocol version, for example `HTTP/1.1`
- The other lines are **request header fields**
- Each line is terminated by a CR/LF (ASCII: 13 10)

On this slide, you can see a complete, albeit much simplified compared to reality, request that the client sends to the server to query the main website of our faculty. Let's look at this request line by line. The first line contains the command sent to the web server and the local address of the document to be queried. So, in this example, the web browser sends a GET request for the document named `index.html` in the root directory. After the document's name follows the protocol's name, specifically the version of HTTP to be used in the communication. The GET command means the browser wants to load a page from the server. The second line contains the hostname, which is the same name used in the URI. Why this duplication? The simple answer is that a physical computer can also be accessible under several IP addresses or full names. So, a web server can offer documents from different domains under certain circumstances; therefore, the request message is sent again from which domain a document is requested. This second line, in which the hostname is defined, is the first from the request header. The request header contains further data line by line; one could also call this additional parameter, which the client would like to transmit to the server in the request context. In our first example, the request header consists of three lines, often up to 30 or 40 lines long. In line three, the Accept parameter specifies that the client wants to receive a document in a particular format. The value for this parameter is `Text/HTML`, the standardized name for the format of HTML files. In the fourth line, the cache control parameter with the value `"no-cache"` defines the client as wanting to have the latest file version and not a possibly outdated version cached somewhere. We'll go into the concept of caching in great detail later. I just said that we look at the request message line by line because these messages are also constructed line by line, meaning there must be the control sequence CR/LF after each line. An empty line then indicates the end of the header.

16 HTTP Verbs (sometimes called methods)

HTTP Verbs (sometimes called methods)

- GET
 - Request a document or (in general) a resource
 - The most used HTTP verb
- POST
 - Send data from the client to the server
 - Used to transmit form data
- PUT to update resources (we postpone that)
- DELETE to delete resources (we postpone that)

When a web browser wants to request a document from a web server, it sends a request message using the GET command, as we have just seen. However, this is one of many commands that may be used in a request message. By the way, the technically correct term for "command" is HTTP verb or HTTP method. In addition to the GET request for retrieving documents, there are also POST requests, which the client can use to send data to the server. Surely, you have already entered data in a form on a website; these are then usually transmitted to the server in the form of such a POST request. There is also the HTTP verb PUT, which can be used to modify documents on the server, and the HTTP verb DELETE, which can be used to delete documents on the server. These two HTTP verbs are rarely used in web pages, but we will see later that they are important when implementing more complex distributed systems.

17 Some examples for HTTP request header fields

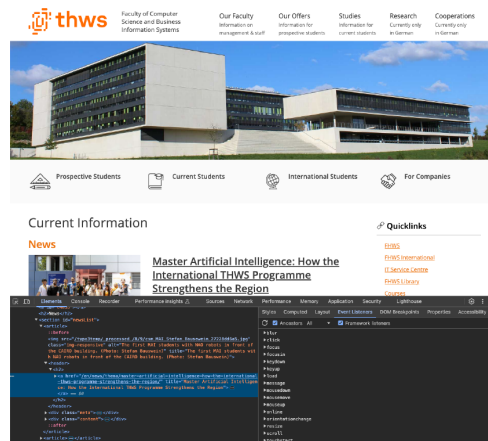
Some examples for HTTP request header fields

- **Accept:** the media type that the clients would like to get
- **Accept-Encoding:** Acceptable compression methods
- **Authorization:** User name and password
- **User-Agent:** The type of the Web browser

I just mentioned that, in reality, the header of such an HTTP request is much longer than in our first small example. At this point, a few more parameters are typically transmitted within the header area from the client to the server. We have already discussed the first parameter, **Accept**, which the client uses to specify the format of the document to be retrieved by the server. The client uses the second parameter, **Accept-Encoding**, to specify whether the server can transmit the document in compressed form in the response message. The third parameter, **Authorization**, must be used if access to the queried document is only allowed to specific users. With this parameter, the client could, for example, transmit the user's name and password. With the fourth parameter, the client sends a lot of information about the browser used, i.e., about itself, so to speak, and sometimes a few pieces of device information. Based on this value, the server can decide, for example, whether the request was just issued from a desktop computer or a mobile device and adjust its response accordingly.

18 Use your browser to inspect HTTP messages

Use your browser to inspect HTTP messages



- Open the developer console of your Web browser

HTTP messages are text-based and, therefore, easy for humans to read and understand. There are many tools to view HTTP messages; for example, every web browser offers developers a specific mode to view every request the web browser sends to the server with the content of the request, the corresponding response message, and the time history. Another tool that I would like to recommend to you and that we will use from time to time in the course is called Postman. Please install this application on your computer, familiarize yourself with its operation, make some requests to a web server, and look at the contents of the HTTP messages.

19 Screencast 1

Screencast 1

Just a few comments about this app Postman. You can start by entering here the URL of a website. And we use the website of our faculty here. What we send is a GET request. We want to load the document here. And with Postman you can now inspect, so to speak, all aspects of the HTTP protocol. So here in this area you can define what kind of authorization, for example, should be used. What headers should be sent. You can also send a buddy as part of a POST request or PUT request. And the response is shown here. You can switch between different kind of views, like the raw view of the HTML page here. Or like a pretty printing of the HTML page. You can also activate a preview, but this is not, let's say, it's similar, of course, to what you see in the browser, but it's not really comparable. But what will be interesting for us most will be the headers. So here you can see the status code of this request is 200. So it was successful. Here you can see the time. So it took about 1.3 seconds for the client to load this web page. And for this first request 14 kilobyte of data had to be transferred. Then you can see here some header information, like date, server, content language and so on. Encoding the length here. And content type.

20 Response of the Web server

Response of the Web server

- The Web server receives the HTTP **request**
- It replies with an HTTP **response** message
- The response message has the following structure:

```
1 HTTP/1.1 200 OK
2 Date: Mon, 17 Aug 2024 14:08:16 GMT
3 Server: Apache/2.2.22 (Ubuntu)
4 Content-Type: text/html; charset=utf-8
5
6 <!DOCTYPE html
7   PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
8   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

Back to our example: the first request to our faculty web server. So, the request has been received by the web server and is now processed. The web browser wanted to initiate the file transfer with the name `Index.html` in the root directory with the request; the webserver looks in its local directory for the file with the matching name and responds if this was a successful message to the client. The structure of this response message is again straightforward; it is text-oriented and, at least at the beginning of the message, also divided into individual lines.

21 Structure of the Response Message

Structure of the Response Message

```
1 HTTP/1.1 200 OK
2 Date: Mon, 17 Aug 2024 14:08:16 GMT
3 Server: Apache/2.2.22 (Ubuntu)
4 Content-Type: text/html; charset=utf-8
5
6 <!DOCTYPE html
7   PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
8   "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
9   <html xmlns="http://www.w3.org/1999/xhtml" xml:lang="de" lang="de"
   ">
```

- The first line is named **status line**
- It contains the **status code**
- The other lines are **response header fields**
- The **response body** contains the document
- Each line is terminated by a CR/LF (ASCII: 13 10)

The first line starts with the name of the protocol, followed by the so-called status code; in this example, the status code has the value 200, and at the end of the first line, it says OK, which is just a human-readable description of the status code. The status code 200 means that the client's request could be processed successfully by the server, and in the further course of this answer message is now the requested document. Starting from the second line, identical to the request structure, additional data or parameters are part of the answer. Line 2 contains the date when the request was processed. The name of the software that processed the request is disclosed in the third line. In this case, even the name of the operating system is revealed. In the fourth line, the web server transmits the information in which file format the answer is transmitted and the character encoding – in this case, UTF-8. Line 5 is the conclusion of the header area, therefore an empty line, and in line 6, the contents of the requested file begin. Each line must be marked in the header area by an end of line character – i.e., CR/LF. Of course, this does not apply in the content area of the response message. A text file was queried. Accordingly, the content area of the response message contains a human-readable file, in this case in HTML format. However, it would also be permissible to transmit a response message with binary data in the content area, for example, images or PDF files.

22 HTTP Status Codes

HTTP Status Codes

- **1xx**: Informational, request is still processed
- **2xx**: Success, the request was processed successfully, and the response is given
- **3xx**: Redirection, the client must send another request to complete the request
- **4xx**: Client Error, the request was not correct
- **5xx**: Server Error, the request was correct, but the server was not able to handle it

In this example, the server successfully processed the request from the web browser and used status code 200 in its response message. This is the most frequent status code, and it indicates that there was no error. However, not all requests are successful. Accordingly, there are many other status codes that the server transmits back to the client in the response message to inform it about these different situations. The status codes are first divided into the following five groups. Status codes that start with a 1, theoretically anything from 100-199, inform the client that the request has so far only been partially received by the server, for example, only the header and that the client should now proceed to send the rest of the request over the same connection. Status codes that start with two generally indicate that the server successfully processed the client's request. The status code 200, shown in the example, is the most frequently used status code, which means that the requested document is part of the content area of the response. Other status codes also indicate that the request was successfully processed, but the body does not contain the document for reasons that are not important. Nevertheless, the request was successful. Status codes that start with a 3 inform the client that the request was correct in principle but that the requested document is no longer available at this address. In the response, the server tells the client a new request should be made under a new address. Status codes starting with a four always indicate an error in the client's request. The most common error message you have probably seen several times is the error code 404, which informs the client that no document is available at the specified address. Clients can make many mistakes with their requests, so there are a lot of status codes, especially in the 400 range, which we will have to look at later. With a status code from this range, the server informs the client that it should or must modify its request so that it becomes successful. It thus tells the client that it is not helpful to send precisely

the same request again. In contrast, status codes starting with 5 inform the client that the server had a problem while processing a valid request. For example, the database was unavailable when this request was being processed, so the entire request could not be handled now. From such a status code, the client knows that his request was, in principle, correct, and he could, for example, send it again after a waiting period.

23 Examples of HTTP response headers

Examples of HTTP response headers

- **Content-Type:**
Type of the document provided, e.g., `text/html`
- **Content-Length:**
Size of the response body in bytes
- **Last-Modified:**
Timestamp of the last modification of this document

In the response's header, the server transmits more data or parameters to the client, as we have seen briefly above. Here are a few more examples. The parameter "Content-Type" indicates the data format in which the server sent the response. This information may be redundant because the client has already sent the request in the data format where he wants the answer. However, the client can specify several different data formats that suit him. The server can then select a data format from the offered set. This data format chosen must then be explicitly specified again in the response message. The parameter "Content-Length" gives the client information about the length of the content area or, in principle, the size of the response, but without the header area. This information can be beneficial for the client in preparing itself to process the response message. The last parameter, "last modified," gives the client the information when the requested file was updated for the last time. This gives the client an indication of how fresh the data in the response message is. It can later, but we'll come back to this in detail and specify in a later request to the server that it only wants a response if the file has changed.

24 HTTP Query Parameters

HTTP Query Parameters

- URLs can contain parameters
- `.../ping?key=value&anotherkey=anothervalue`

When we have just looked at the structure of a URI, i.e., an address, we have already briefly learned the possibility that further parameters are specified as part of the request but already in the address. A question mark separates these parameters from the path or file name in the address, which always consists of a parameter name and the corresponding value. An ampersand separates these pairs. At this point, we now use the term parameter with two meanings. On the one hand, we have just talked about parameters used in the header of a request or response message. Here, we are talking about parameters that are part of the address. Technically, the parameters from the address could also be implemented as parameters in the header of the request message. However, the meanings of these two types of parameters are different. Parameters in the request's header inform the server about relevant settings when processing the request. Parameters in the address are used to control the selection of the document or, more generally, the selection of the data that the server should send back to the client. We briefly extend the idea that HTTP requests should only transfer documents and consider an example where the client asks for data from a database. Then, you can think of these parameters as part of the address, like filters that could be passed to the database to limit the result set.

25 The most important feature: HTTP is state-less

The most important feature: HTTP is state-less

- TCP is a state-full protocol.
- HTTP is by specification a **state-less** protocol
- Each request-response is processed independently
- Clients must send **all** information necessary for processing a request

Before we end with the first introduction to the HTTP protocol, I would like to talk about a very important or even **the** most important property of HTTP. HTTP is a so-called stateless protocol. This means that every single communication step between client and server in the form of a request message, which is then answered by the server with a response message, is a self-contained process. The server does not keep anything in mind about the processing of such a request and, therefore, cannot know which requests the same client has already sent before. Thus, the server does not remember any state from the client; therefore, for each request, it needs all the information necessary to process that request. Technically, in some situations, there is a requirement to circumvent this limitation. For example, with the help of so-called session IDs generated by the server and then constantly transmitted between client and server. With such a session ID, the server can detect two requests from an identical client. However, using such session IDs has technical disadvantages; therefore, we should refrain from using them.

26 Summary

Summary

- You know about the history of the Internet
- You know the basics of HTML
- You know the structure of HTTP messages

In this unit, we explored the fundamentals of HTTP, the protocol that powers the World Wide Web. We began with a brief history of the internet, tracing its roots back to the launch of Sputnik in 1957, which led to the development of the ARPANET and, eventually, the modern internet. We then moved on to Tim Berners-Lee's creation of the World Wide Web in 1989, focusing on his introduction of the HTTP protocol and HTML, revolutionizing how documents are shared and accessed online. We learned that the World Wide Web is today's largest distributed system, operating on a client-server model. Web browsers act as clients, sending requests to web servers, which respond by serving HTML documents. We examined the structure of these interactions, particularly through GET requests, where browsers request documents from servers, and servers respond with status codes and requested content. A key takeaway is that HTTP is a stateless protocol—each interaction between client and server is independent, with no memory of previous requests. While session IDs can track state across requests, it's generally better to avoid them due to their complexity. Finally, we touched on the importance of hypertext and hyperlinks, which link documents across the web, creating a vast, interconnected network of information. This understanding of HTTP provides a solid foundation for further exploration into web development and network communication in future units.

27 Literature

Literature

- Katie Hafner et al: Die Anfänge des Internet. 2. Auflage, dpunkt Verlag, 2008.
- Christoph Meinel und Harald Sack: WWW. Springer Verlag, 2012.